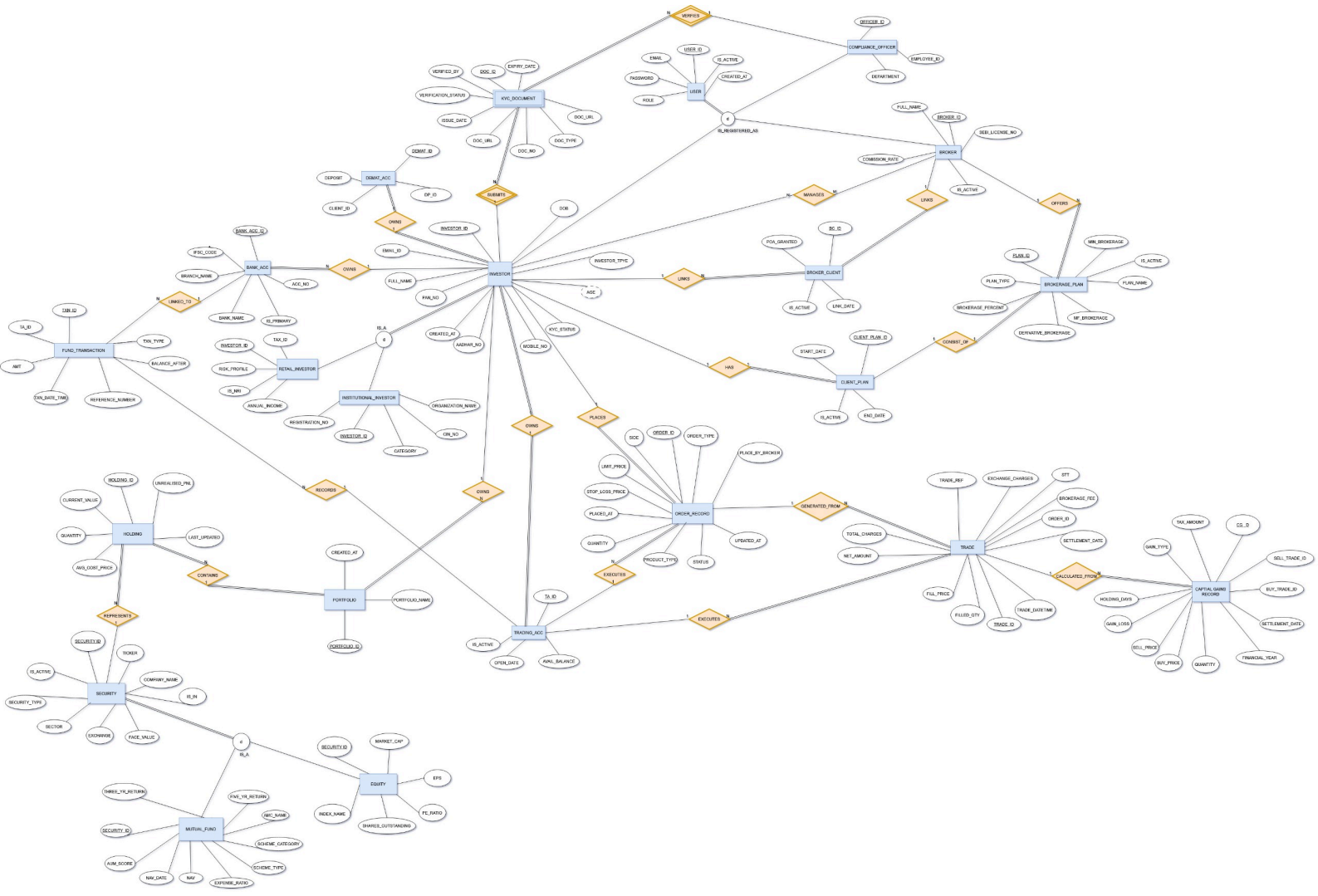
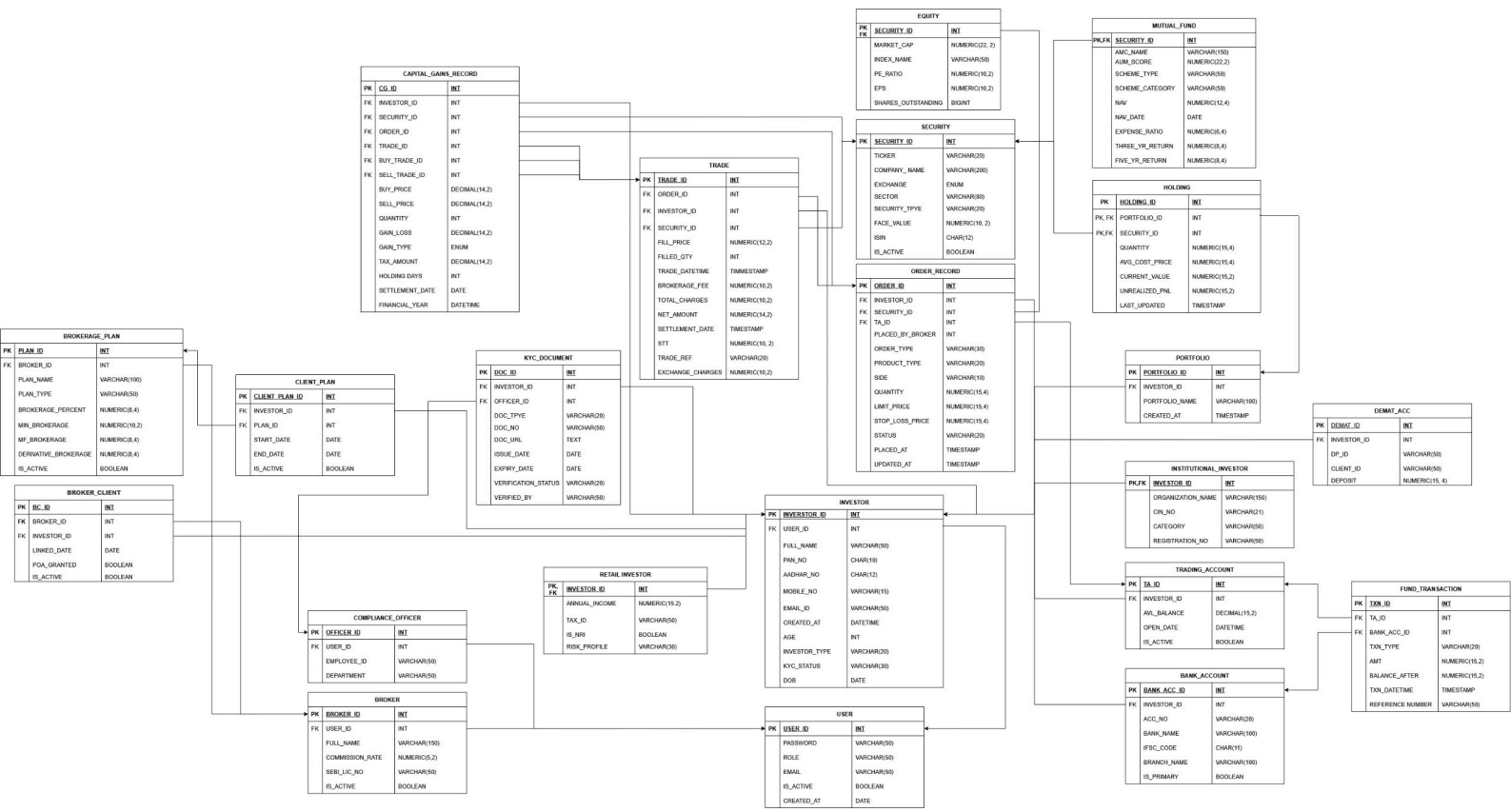

VERIFIES
OFFICER_ID
COMPLIANCE_OFFICER
EMPLOYEE_ID
DEPARTMENT
USER_ID
EMAIL
IS_ACTIVE
CREATED_AT
PASSWORD
ROLE
USER
VERIFIED_BY
DOC_ID
EXPIRY_DATE
VERIFICATION_STATUS
ISSUE_DATE
DOC_URL
DOC_NO
DOC_TYPE
KYC_DOCUMENT
IS_REGISTERED_AS
FULL_NAME
BROKER_ID
SEBI_LICENSE_NO
COMMISSION_RATE
BROKER
DEMAT_ID
DEPOSIT
CLIENT_ID
DP_ID
DEMAT_ACC
SUBMITS
MANAGES
LINKS
IS_ACTIVE
OFFERS
OWNS
INVESTOR_ID
DOB
BANK_ACC_ID
IFSC_CODE
BRANCH_NAME
BANK_ACC
OWNS
EMAIL_ID
FULL_NAME
PAN_NO
ACC_NO
INVESTOR
INVESTOR_TYPE
LINKS
BC_ID
POA_GRANTED
BROKER_CLIENT
IS_ACTIVE
LINK_DATE
MIN_BROKERAGE
PLAN_ID
IS_ACTIVE
PLAN_TYPE
BROKERAGE_PLAN
PLAN_NAME
BROKERAGE_PERCENT
MF_BROKERAGE
DERIVATIVE_BROKERAGE
LINKED_TO
BANK_NAME
IS_PRIMARY
TXN_ID
TA_ID
FUND_TRANSACTION
TXN_TYPE
AMT
BALANCE_AFTER
TXN_DATE_TIME
REFERENCE_NUMBER
IS_A
d
INVESTOR_ID
TAX_ID
RISK_PROFILE
IS_NRI
ANNUAL_INCOME
RETAIL_INVESTOR
REGISTRATION_NO
ORGANIZATION_NAME
INVESTOR_ID
CIN_NO
CATEGORY
INSTITUTIONAL_INVESTOR
CREATED_AT
AADHAAR_NO
MOBILE_NO
KYC_STATUS
HAS
CLIENT_PLAN_ID
START_DATE
IS_ACTIVE
END_DATE
CLIENT_PLAN
CONNECT_OF
PLACES
OWNS
SIDE
ORDER_ID
ORDER_TYPE
PLACE_BY_BROKER
LIMIT_PRICE
STOP_LOSS_PRICE
PLACED_AT
QUANTITY
PRODUCT_TYPE
STATUS
UPDATED_AT
ORDER_RECORD
GENERATED_FROM
TRADE_REF
EXCHANGE_CHARGES
STT
BROKERAGE_FEE
ORDER_ID
SETTLEMENT_DATE
TOTAL_CHARGES
NET_AMOUNT
TRADE
FILL_PRICE
FILLED_QTY
TRADE_DATETIME
TRADE_ID
CALCULATED_FROM
TAX_AMOUNT
CG_ID
GAIN_TYPE
SELL_TRADE_ID
BUY_TRADE_ID
HOLDING_DAYS
GAIN_LOSS
SELL_PRICE
BUY_PRICE
QUANTITY
SETTLEMENT_DATE
FINANCIAL_YEAR
CAPITAL_GAINS_RECORD
EXECUTES
TA_ID
IS_ACTIVE
OPEN_DATE
AVAIL_BALANCE
TRADING_ACC
EXECUTES
RECORDS
OWNS
HOLDING_ID
UNREALISED_PNL
CURRENT_VALUE
QUANTITY
LAST_UPDATED
AVG_COST_PRICE
HOLDING
CONTAINS
REPRESENTS
CREATED_AT
PORTFOLIO_NAME
PORTFOLIO_ID
PORTFOLIO
SECURITY_ID
TICKER
IS_ACTIVE
COMPANY_NAME
SECURITY_TYPE
IS_IN
SECTOR
EXCHANGE
FACE_VALUE
SECURITY
d
IS_A
SECURITY_ID
MARKET_CAP
EPS
INDEX_NAME
PE_RATIO
SHARES_OUTSTANDING
EQUITY
THREE_YR_RETURN
FIVE_YR_RETURN
AMC_NAME
SECURITY_ID
SCHEME_CATEGORY
AUM_SCORE
SCHEME_TYPE
NAV_DATE
NAV
EXPENSE_RATIO
MUTUAL_FUND


EQUITY
PK FK SECURITY_ID INT
MARKET_CAP NUMERIC(22, 2)
INDEX_NAME VARCHAR(50)
PE_RATIO NUMERIC(10,2)
EPS NUMERIC(10,2)
SHARES_OUTSTANDING BIGINT
MUTUAL_FUND
PK,FK SECURITY_ID INT
AMC_NAME VARCHAR(150)
AUM_SCORE NUMERIC(22,2)
SCHEME_TYPE VARCHAR(50)
SCHEME_CATEGORY VARCHAR(50)
NAV NUMERIC(12,4)
NAV_DATE DATE
EXPENSE_RATIO NUMERIC(8,4)
THREE_YR_RETURN NUMERIC(8,4)
FIVE_YR_RETURN NUMERIC(8,4)
CAPITAL_GAINS_RECORD
PK CG_ID INT
FK INVESTOR_ID INT
FK SECURITY_ID INT
FK ORDER_ID INT
FK TRADE_ID INT
FK BUY_TRADE_ID INT
FK SELL_TRADE_ID INT
BUY_PRICE DECIMAL(14,2)
SELL_PRICE DECIMAL(14,2)
QUANTITY INT
GAIN_LOSS DECIMAL(14,2)
GAIN_TYPE ENUM
TAX_AMOUNT DECIMAL(14,2)
HOLDING DAYS INT
SETTLEMENT_DATE DATE
FINANCIAL_YEAR DATETIME
SECURITY
PK SECURITY_ID INT
TICKER VARCHAR(20)
COMPANY_NAME VARCHAR(200)
EXCHANGE ENUM
SECTOR VARCHAR(80)
SECURITY_TPYE VARCHAR(20)
FACE_VALUE NUMERIC(10, 2)
ISIN CHAR(12)
IS_ACTIVE BOOLEAN
HOLDING
PK HOLDING_ID INT
PK, FK PORTFOLIO_ID INT
PK,FK SECURITY_ID INT
QUANTITY NUMERIC(15,4)
AVG_COST_PRICE NUMERIC(15,4)
CURRENT_VALUE NUMERIC(15,2)
UNREALIZED_PNL NUMERIC(15,2)
LAST_UPDATED TIMESTAMP
TRADE
PK TRADE_ID INT
FK ORDER_ID INT
FK INVESTOR_ID INT
FK SECURITY_ID INT
FILL_PRICE NUMERIC(12,2)
FILLED_QTY INT
TRADE_DATETIME TIMMESTAMP
BROKERAGE_FEE NUMERIC(10,2)
TOTAL_CHARGES NUMERIC(10,2)
NET_AMOUNT NUMERIC(14,2)
SETTLEMENT_DATE TIMESTAMP
STT NUMERIC(10, 2)
TRADE_REF VARCHAR(20)
EXCHANGE_CHARGES NUMERIC(10,2)
ORDER_RECORD
PK ORDER_ID INT
FK INVESTOR_ID INT
FK SECURITY_ID INT
FK TA_ID INT
PLACED_BY_BROKER INT
ORDER_TYPE VARCHAR(30)
PRODUCT_TYPE VARCHAR(20)
SIDE VARCHAR(10)
QUANTITY NUMERIC(15,4)
LIMIT_PRICE NUMERIC(15,4)
STOP_LOSS_PRICE NUMERIC(15,4)
STATUS VARCHAR(20)
PLACED_AT TIMESTAMP
UPDATED_AT TIMESTAMP
BROKERAGE_PLAN
PK PLAN_ID INT
FK BROKER_ID INT
PLAN_NAME VARCHAR(100)
PLAN_TYPE VARCHAR(50)
BROKERAGE_PERCENT NUMERIC(8,4)
MIN_BROKERAGE NUMERIC(10,2)
MF_BROKERAGE NUMERIC(8,4)
DERIVATIVE_BROKERAGE NUMERIC(8,4)
IS_ACTIVE BOOLEAN
CLIENT_PLAN
PK CLIENT_PLAN_ID INT
FK INVESTOR_ID INT
FK PLAN_ID INT
START_DATE DATE
END_DATE DATE
IS_ACTIVE BOOLEAN
KYC_DOCUMENT
PK DOC_ID INT
FK INVESTOR_ID INT
FK OFFICER_ID INT
DOC_TPYE VARCHAR(20)
DOC_NO VARCHAR(50)
DOC_URL TEXT
ISSUE_DATE DATE
EXPIRY_DATE DATE
VERIFICATION_STATUS VARCHAR(20)
VERIFIED_BY VARCHAR(50)
PORTFOLIO
PK PORTFOLIO_ID INT
FK INVESTOR_ID INT
PORTFOLIO_NAME VARCHAR(100)
CREATED_AT TIMESTAMP
DEMAT_ACC
PK DEMAT_ID INT
FK INVESTOR_ID INT
DP_ID VARCHAR(50)
CLIENT_ID VARCHAR(50)
DEPOSIT NUMERIC(15, 4)
BROKER_CLIENT
PK BC_ID INT
FK BROKER_ID INT
FK INVESTOR_ID INT
LINKED_DATE DATE
POA_GRANTED BOOLEAN
IS_ACTIVE BOOLEAN
INVESTOR
PK INVERSTOR_ID INT
FK USER_ID INT
FULL_NAME VARCHAR(50)
PAN_NO CHAR(10)
AADHAR_NO CHAR(12)
MOBILE_NO VARCHAR(15)
EMAIL_ID VARCHAR(50)
CREATED_AT DATETIME
AGE INT
INVESTOR_TYPE VARCHAR(20)
KYC_STATUS VARCHAR(30)
DOB DATE
INSTITUTIONAL_INVESTOR
PK,FK INVESTOR_ID INT
ORGANIZATION_NAME VARCHAR(150)
CIN_NO VARCHAR(21)
CATEGORY VARCHAR(50)
REGISTRATION_NO VARCHAR(50)
RETAIL INVESTOR
PK, FK INVESTOR_ID INT
ANNUAL_INCOME NUMERIC(15,2)
TAX_ID VARCHAR(50)
IS_NRI BOOLEAN
RISK_PROFILE VARCHAR(30)
TRADING_ACCOUNT
PK TA_ID INT
FK INVESTOR_ID INT
AVL_BALANCE DECIMAL(15,2)
OPEN_DATE DATETIME
IS_ACTIVE BOOLEAN
FUND_TRANSACTION
PK TXN_ID INT
FK TA_ID INT
FK BANK_ACC_ID INT
TXN_TYPE VARCHAR(20)
AMT NUMERIC(15,2)
BALANCE_AFTER NUMERIC(15,2)
TXN_DATETIME TIMESTAMP
REFERENCE NUMBER VARCHAR(50)
COMPLIANCE_OFFICER
PK OFFICER_ID INT
FK USER_ID INT
EMPLOYEE_ID VARCHAR(50)
DEPARTMENT VARCHAR(50)
BANK_ACCOUNT
PK BANK_ACC_ID INT
FK INVESTOR_ID INT
ACC_NO VARCHAR(20)
BANK_NAME VARCHAR(100)
IFSC_CODE CHAR(11)
BRANCH_NAME VARCHAR(100)
IS_PRIMARY BOOLEAN
BROKER
PK BROKER_ID INT
FK USER_ID INT
FULL_NAME VARCHAR(150)
COMMISSION_RATE NUMERIC(5,2)
SEBI_LIC_NO VARCHAR(50)
IS_ACTIVE BOOLEAN
USER
PK USER_ID INT
PASSWORD VARCHAR(50)
ROLE VARCHAR(50)
EMAIL VARCHAR(50)
IS_ACTIVE BOOLEAN
CREATED_AT DATE

StockVault – Functional Dependency & BCNF Analysis

Subject: Database Systems | Topic: BCNF Verification & Decomposition

0. Notation Used

Quick reference for symbols used throughout this document:

Symbol	What it means
PK	Primary Key – always a superkey
CK	Candidate Key – smallest possible superkey
FD	Functional Dependency
→	Functionally determines
■ (before FD)	Does NOT hold in both directions
✓	BCNF condition is satisfied
✗	BCNF condition is violated
Non-trivial FD	X → Y where Y is not a subset of X

For each table we: list all attributes, list all non-trivial FDs, find all candidate keys, check BCNF, and fix any violations with a lossless decomposition.

Quick Summary

Item	Count
Total Tables	22
Tables in BCNF	17 ✓
Tables with violations	5 ✗ (Tables 3, 8, 17, 20, 21)
Total violations found	7 (across 5 tables)
All violations fixed?	Yes – all decompositions are lossless

Table 1 – USER

Attributes: USER_ID (PK), EMAIL (CK), PASSWORD, ROLE, IS_ACTIVE, CREATED_AT

Candidate Keys: {USER_ID}, {EMAIL}

FD	Dependency	Status
FD1	USER_ID → EMAIL, PASSWORD, ROLE, IS_ACTIVE, CREATED_AT	✓ (CK1)
FD2	EMAIL → USER_ID, PASSWORD, ROLE, IS_ACTIVE, CREATED_AT	✓ (CK2)

✓ In BCNF – both FDs have a superkey on the left-hand side.

Table 2 – COMPLIANCE_OFFICER

Attributes: OFFICER_ID (PK), EMPLOYEE_ID (CK), DEPARTMENT, USER_ID

Candidate Keys: {OFFICER_ID}, {EMPLOYEE_ID}

FD	Dependency	Status
FD1	OFFICER_ID → EMPLOYEE_ID, DEPARTMENT, USER_ID	✓ (CK1)
FD2	EMPLOYEE_ID → OFFICER_ID, DEPARTMENT, USER_ID	✓ (CK2)

✓ In BCNF – both FDs have a superkey on the LHS.

Table 3 – INVESTOR ✗ VIOLATION

Attributes: INVESTOR_ID (PK), USER_ID, FULL_NAME, EMAIL_ID, PAN_NO (CK), MOBILE_NO, DOB, AGE, AADHAR_NO (CK), KYC_STATUS, CREATED_AT, INVESTOR_TYPE

Candidate Keys: {INVESTOR_ID}, {PAN_NO}, {AADHAR_NO}

FD	Dependency	Status
FD1	INVESTOR_ID → all other attributes	✓ (CK1)
FD2	PAN_NO → INVESTOR_ID (and transitively all others)	✓ (CK2)
FD3	AADHAR_NO → INVESTOR_ID (and transitively all others)	✓ (CK3)
FD4 ■	DOB → AGE	✗ VIOLATION

Why it's a violation

AGE is just a derived attribute – it's completely computed from DOB (and today's date). Storing both in the same table means if someone fixes a wrong DOB, they also have to manually update AGE, otherwise the row becomes inconsistent. And DOB alone is definitely not a superkey – lots of investors share the same birthday. So FD4 breaks BCNF.

Fix – Lossless Decomposition

Just remove the AGE column. Compute it dynamically in queries using PostgreSQL's date functions. After removing AGE, only FD1/FD2/FD3 remain and all their LHS are candidate keys, so BCNF is satisfied. This is technically a trivial lossless decomposition (we removed a column, not split the table).

Corrected DDL:

```
CREATE TABLE INVESTOR (
  INVESTOR_ID    SERIAL PRIMARY KEY,
  USER_ID        INT REFERENCES "USER"(USER_ID) ON DELETE SET NULL,
  FULL_NAME      VARCHAR(50) NOT NULL,
```

```
  EMAIL_ID       VARCHAR(50),
  PAN_NO         CHAR(10) UNIQUE,
  MOBILE_NO      VARCHAR(15),
  DOB            DATE,   -- age = EXTRACT(YEAR FROM AGE(DOB))
  AADHAR_NO      CHAR(12) UNIQUE,
  KYC_STATUS     VARCHAR(30),
  CREATED_AT     DATE DEFAULT CURRENT_DATE,
  INVESTOR_TYPE  VARCHAR(20)
    CHECK (INVESTOR_TYPE IN ('RETAIL','HNI','INSTITUTIONAL'))
);
```

Table 4 – KYC_DOCUMENT

Attributes: DOC_ID (PK), DOC_NO (CK), DOC_TYPE, DOC_URL, ISSUE_DATE, EXPIRY_DATE, VERIFICATION_STATUS, OFFICER_ID, INVESTOR_ID, VERIFIED_BY

Candidate Keys: {DOC_ID}, {DOC_NO}

FD	Dependency	Status
FD1	DOC_ID → all other attributes	✓ (CK1)
FD2	DOC_NO → DOC_ID (and all others – UNIQUE constraint)	✓ (CK2)

✓ In BCNF – all FDs have a superkey on the LHS.

Table 5 – RETAIL_INVESTOR

Attributes: INVESTOR_ID (PK/FK), TAX_ID, RISK_PROFILE, IS_NRI, ANNUAL_INCOME

Candidate Keys: {INVESTOR_ID}

FD	Dependency	Status
FD1	INVESTOR_ID → TAX_ID, RISK_PROFILE, IS_NRI, ANNUAL_INCOME	✓ (CK1)

✓ In BCNF – single FD, LHS is the primary key. This is an ISA specialization of INVESTOR.

Table 6 – INSTITUTIONAL_INVESTOR

Attributes: INVESTOR_ID (PK/FK), ORGANIZATION_NAME, CIN_NO (CK), REGISTRATION_NO, CATEGORY

Candidate Keys: {INVESTOR_ID}, {CIN_NO}

FD	Dependency	Status
FD1	INVESTOR_ID → ORGANIZATION_NAME, CIN_NO, REGISTRATION_NO, CATEGORY	✓ (CK1)
FD2	CIN_NO → INVESTOR_ID (UNIQUE constraint)	✓ (CK2)

✓ In BCNF – both FDs have superkeys on the LHS.

Table 7 – DEMAT_ACC

Attributes: DEMAT_ID (PK), CLIENT_ID, DP_ID, DEPOSIT, INVESTOR_ID

Candidate Keys: {DEMAT_ID}

FD	Dependency	Status
FD1	DEMAT_ID → CLIENT_ID, DP_ID, DEPOSIT, INVESTOR_ID	✓ (CK1)

✓ In BCNF – single FD, LHS is the primary key.

Table 8 – BANK_ACC ✗ VIOLATION

Attributes: BANK_ACC_ID (PK), ACC_NO (CK), BANK_NAME, BRANCH_NAME, IFSC_CODE, IS_PRIMARY, INVESTOR_ID

Candidate Keys: {BANK_ACC_ID}, {ACC_NO}

FD	Dependency	Status
FD1	BANK_ACC_ID → all other attributes	✓ (CK1)
FD2	ACC_NO → BANK_ACC_ID (and all others)	✓ (CK2)
FD3 ■	IFSC_CODE → BANK_NAME, BRANCH_NAME	✗ VIOLATION

Why it's a violation

The RBI IFSC code uniquely identifies a specific bank branch, so IFSC_CODE → BANK_NAME and BRANCH_NAME in the real world. But IFSC_CODE is not a superkey of BANK_ACC – many different account holders can be at the same branch. So the same bank name and branch name gets stored redundantly for every account at that branch. If a branch gets renamed, you'd have to update every single row – classic update anomaly.

Fix – Lossless Decomposition

Split into two tables: R1 holds IFSC → BANK_NAME, BRANCH_NAME. R2 keeps the account info with IFSC_CODE as a foreign key. Join on IFSC_CODE to get the original data back (lossless).

R1: BANK_BRANCH(IFSC_CODE [PK], BANK_NAME, BRANCH_NAME)

R2: BANK_ACC(BANK_ACC_ID [PK], ACC_NO [CK], IFSC_CODE [FK], IS_PRIMARY, INVESTOR_ID)

Corrected DDL:

```
CREATE TABLE BANK_BRANCH (
  IFSC_CODE   CHAR(11) PRIMARY KEY,
```

```
  BANK_NAME   VARCHAR(100) NOT NULL,
  BRANCH_NAME VARCHAR(100)
);

CREATE TABLE BANK_ACC (
  BANK_ACC_ID SERIAL PRIMARY KEY,
  ACC_NO      VARCHAR(20) UNIQUE NOT NULL,
  IFSC_CODE   CHAR(11) REFERENCES BANK_BRANCH(IFSC_CODE),
  IS_PRIMARY  BOOLEAN DEFAULT FALSE,
  INVESTOR_ID INT REFERENCES INVESTOR(INVESTOR_ID) ON DELETE CASCADE
);
```

Table 9 – FUND_TRANSACTION

Attributes: TXN_ID (PK), TA_ID, BANK_ACC_ID, TXN_TYPE, AMT, BALANCE_AFTER, TXN_DATE_TIME, REFERENCE_NUMBER

Candidate Keys: {TXN_ID}, {REFERENCE_NUMBER}

FD	Dependency	Status
FD1	TXN_ID → all other attributes	✓ (CK1)
FD2	REFERENCE_NUMBER → TXN_ID (UTR/IMPS ref is globally unique)	✓ (CK2)

✓ In BCNF – all FDs have superkeys on the LHS.

Table 10 – BROKER

Attributes: BROKER_ID (PK), USER_ID (CK), FULL_NAME, SEBI_LICENSE_NO (CK), COMMISSION_RATE, IS_ACTIVE

Candidate Keys: {BROKER_ID}, {USER_ID}, {SEBI_LICENSE_NO}

FD	Dependency	Status
FD1	BROKER_ID → all other attributes	✓ (CK1)
FD2	USER_ID → BROKER_ID (UNIQUE NOT NULL)	✓ (CK2)
FD3	SEBI_LICENSE_NO → BROKER_ID (UNIQUE NOT NULL)	✓ (CK3)

✓ In BCNF – three candidate keys, all FDs have a CK on the LHS.

Table 11 – BROKERAGE_PLAN

Attributes: PLAN_ID (PK), PLAN_NAME, PLAN_TYPE, MIN_BROKERAGE, BROKERAGE_PERCENT, DERIVATIVE_BROKERAGE, MF_BROKERAGE, IS_ACTIVE, BROKER_ID

Candidate Keys: {PLAN_ID}

FD	Dependency	Status
FD1	PLAN_ID → all other attributes	✓ (CK1)

✓ In BCNF – single FD, LHS is the primary key. Note: (BROKER_ID, PLAN_NAME) could be a composite candidate key if plan names are unique per broker, but the schema doesn't enforce it so we don't treat it as one.

Table 12 – BROKER_CLIENT

Attributes: BC_ID (PK), INVESTOR_ID, BROKER_ID, POA_GRANTED, IS_ACTIVE, LINK_DATE

Candidate Keys: {BC_ID}, {INVESTOR_ID, BROKER_ID}

FD	Dependency	Status
FD1	BC_ID → INVESTOR_ID, BROKER_ID, POA_GRANTED, IS_ACTIVE, LINK_DATE	✓ (CK1)
FD2	(INVESTOR_ID, BROKER_ID) → BC_ID, POA_GRANTED, IS_ACTIVE, LINK_DATE	✓ (CK2)

✓ In BCNF – both candidate keys appear as LHS of all non-trivial FDs.

Table 13 – PORTFOLIO

Attributes: PORTFOLIO_ID (PK), PORTFOLIO_NAME, CREATED_AT, INVESTOR_ID

Candidate Keys: {PORTFOLIO_ID}

FD	Dependency	Status
FD1	PORTFOLIO_ID → PORTFOLIO_NAME, CREATED_AT, INVESTOR_ID	✓ (CK1)

✓ In BCNF. Note: INVESTOR_ID does NOT determine PORTFOLIO_NAME here because one investor can have many portfolios with different names. So there's no hidden violation.

Table 14 – SECURITY

Attributes: SECURITY_ID (PK), TICKER (CK), COMPANY_NAME, EXCHANGE, SECTOR, SECURITY_TYPE, FACE_VALUE, ISIN (CK), IS_ACTIVE

Candidate Keys: {SECURITY_ID}, {TICKER}, {ISIN}

FD	Dependency	Status
FD1	SECURITY_ID → all other attributes	✓ (CK1)

FD	Dependency	Status
FD2	TICKER → SECURITY_ID (UNIQUE NOT NULL)	✓ (CK2)
FD3	ISIN → SECURITY_ID (ISO 6166, UNIQUE NOT NULL)	✓ (CK3)

✓ In BCNF – three candidate keys, all FDs have a CK on the LHS.

Table 15 – EQUITY

Attributes: SECURITY_ID (PK/FK), MARKET_CAP, INDEX_NAME, PE_RATIO, EPS, SHARES_OUTSTANDING

Candidate Keys: {SECURITY_ID}

FD	Dependency	Status
FD1	SECURITY_ID → MARKET_CAP, INDEX_NAME, PE_RATIO, EPS, SHARES_OUTSTANDING	✓ (CK1)

✓ In BCNF – single FD, LHS is the primary key. ISA specialization of SECURITY.

Table 16 – MUTUAL_FUND

Attributes: SECURITY_ID (PK/FK), AMC_NAME, SCHEME_CATEGORY, SCHEME_TYPE, AUM_SCORE, NAV, NAV_DATE, EXPENSE_RATIO, THREE_YR_RETURN, FIVE_YR_RETURN

Candidate Keys: {SECURITY_ID}

FD	Dependency	Status
FD1	SECURITY_ID → all other attributes	✓ (CK1)

✓ In BCNF – single FD, LHS is the primary key.

Table 17 – HOLDING ✗ VIOLATION

Attributes: HOLDING_ID (PK), PORTFOLIO_ID, SECURITY_ID, QUANTITY, AVG_COST_PRICE, CURRENT_VALUE, UNREALISED_PNL, LAST_UPDATED

Candidate Keys: {HOLDING_ID}, {PORTFOLIO_ID, SECURITY_ID}

FD	Dependency	Status
FD1	HOLDING_ID → all other attributes	✓ (CK1)
FD2	(PORTFOLIO_ID, SECURITY_ID) → HOLDING_ID and all others	✓ (CK2)

FD	Dependency	Status
FD3 ■	{QUANTITY, AVG_COST_PRICE, CURRENT_VALUE} → UNREALISED_PNL	✗ VIOLATION

Why it's a violation

UNREALISED_PNL is a derived value – it's just CURRENT_VALUE minus (AVG_COST_PRICE times QUANTITY). So those three attributes together determine UNREALISED_PNL. But that set is definitely not a superkey – many different holdings can have the same qty, price, and current value numbers. If CURRENT_VALUE gets updated but UNREALISED_PNL doesn't, the stored value is wrong.

Fix – Lossless Decomposition

Remove UNREALISED_PNL and compute it on the fly in queries. Only FD1 and FD2 remain after removal, both with CK on LHS – BCNF satisfied. Trivial lossless decomposition (column removal).

Corrected DDL:

```
CREATE TABLE HOLDING (
  HOLDING_ID      SERIAL PRIMARY KEY,
  PORTFOLIO_ID    INT REFERENCES PORTFOLIO(PORTFOLIO_ID) ON DELETE CASCADE,
  SECURITY_ID     INT REFERENCES SECURITY(SECURITY_ID) ON DELETE CASCADE,
  QUANTITY        NUMERIC(15, 4),
  AVG_COST_PRICE  NUMERIC(15, 4),
  CURRENT_VALUE   NUMERIC(15, 2),
  LAST_UPDATED    TIMESTAMP,
  UNIQUE (PORTFOLIO_ID, SECURITY_ID)
);
-- UNREALISED_PNL = CURRENT_VALUE - (AVG_COST_PRICE * QUANTITY)
```

Table 18 – TRADING_ACC

Attributes: TD_ID (PK), INVESTOR_ID, IS_ACTIVE, AVAIL_BALANCE, OPEN_DATE

Candidate Keys: {TD_ID}

FD	Dependency	Status
FD1	TD_ID → INVESTOR_ID, IS_ACTIVE, AVAIL_BALANCE, OPEN_DATE	✓ (CK1)

✓ In BCNF – single FD, LHS is the primary key.

Table 19 – ORDER_RECORD

Attributes: ORDER_ID (PK), INVESTOR_ID, SECURITY_ID, TA_ID, PLACED_BY_BROKER, SIDE, ORDER_TYPE, PRODUCT_TYPE, QUANTITY, LIMIT_PRICE, STOP_LOSS_PRICE, STATUS, PLACED_AT, UPDATED_AT

Candidate Keys: {ORDER_ID}

FD	Dependency	Status
FD1	ORDER_ID → all other attributes	✓ (CK1)

✓ In BCNF. Note: INVESTOR_ID and SECURITY_ID are just foreign key attributes here – one investor can place many orders and one security can appear in many orders, so neither determines anything else. No hidden violation.

Table 20 – TRADE ✗ VIOLATION

Attributes: TRADE_ID (PK), ORDER_ID, INVESTOR_ID, SECURITY_ID, TRADE_REF (CK), FILL_PRICE, FILLED_QTY, TRADE_DATETIME, SETTLEMENT_DATE, BROKERAGE_FEE, EXCHANGE_CHARGES, STT, TOTAL_CHARGES, NET_AMOUNT

Candidate Keys: {TRADE_ID}, {TRADE_REF}

FD	Dependency	Status
FD1	TRADE_ID → all other attributes	✓ (CK1)
FD2	TRADE_REF → TRADE_ID (UNIQUE NOT NULL)	✓ (CK2)
FD3 ■	ORDER_ID → INVESTOR_ID, SECURITY_ID (via ORDER_RECORD)	✗ VIOLATION 1
FD4 ■	{BROKERAGE_FEE, EXCHANGE_CHARGES, STT} → TOTAL_CHARGES	✗ VIOLATION 2

Violation 1 – ORDER_ID → INVESTOR_ID, SECURITY_ID

Each order has exactly one investor and one security, so ORDER_ID determines both. But one order can generate multiple TRADE rows (partial fills), so ORDER_ID is NOT unique in TRADE – it's not a superkey. Storing INVESTOR_ID and SECURITY_ID in TRADE just repeats them for every partial fill of the same order.

Violation 2 – {BROKERAGE_FEE, EXCHANGE_CHARGES, STT} → TOTAL_CHARGES

TOTAL_CHARGES is just the sum of the three charge columns. It's a derived value – same issue as UNREALISED_PNL. Storing it is redundant and creates inconsistency risk.

Fix – Lossless Decomposition

Remove INVESTOR_ID and SECURITY_ID from TRADE (get them by joining with ORDER_RECORD via ORDER_ID). Also remove TOTAL_CHARGES (compute as BROKERAGE_FEE + EXCHANGE_CHARGES + STT). Both are lossless column-removal decompositions.

Corrected DDL:

```
CREATE TABLE TRADE (
  TRADE_ID          SERIAL PRIMARY KEY,
```

```
  ORDER_ID          INT REFERENCES ORDER_RECORD(ORDER_ID) ON DELETE SET NULL,
  TRADE_REF         VARCHAR(20) UNIQUE,
  FILL_PRICE        NUMERIC(12, 2),
  FILLED_QTY        INT,
  TRADE_DATETIME    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  SETTLEMENT_DATE   DATE,
  BROKERAGE_FEE     NUMERIC(10, 2),
  EXCHANGE_CHARGES  NUMERIC(10, 2),
  STT               NUMERIC(10, 2),
  NET_AMOUNT        NUMERIC(14, 2)
  -- TOTAL_CHARGES = BROKERAGE_FEE + EXCHANGE_CHARGES + STT
  -- INVESTOR_ID via: TRADE -> ORDER_RECORD -> INVESTOR_ID
  -- SECURITY_ID via: TRADE -> ORDER_RECORD -> SECURITY_ID
);
```

Table 21 – CAPITAL_GAINS_RECORD ✗ VIOLATION

Attributes: CG_ID (PK), INVESTOR_ID, SECURITY_ID, ORDER_ID, TRADE_ID, BUY_TRADE_ID, SELL_TRADE_ID, BUY_PRICE, SELL_PRICE, QUANTITY, GAIN_LOSS, GAIN_TYPE, TAX_AMOUNT, HOLDING_DAYS, SETTLEMENT_DATE, FINANCIAL_YEAR

Candidate Keys: {CG_ID}

FD	Dependency	Status
FD1	CG_ID → all other attributes	✓ (CK1)
FD2 ■	SELL_TRADE_ID → INVESTOR_ID, SECURITY_ID, ORDER_ID (via chain)	✗ VIOLATION 1
FD3 ■	HOLDING_DAYS → GAIN_TYPE (< 365 = SHORT_TERM, else LONG_TERM)	✗ VIOLATION 2
FD4 ■	{BUY_PRICE, SELL_PRICE, QUANTITY} → GAIN_LOSS	✗ VIOLATION 3

Violation 1 – SELL_TRADE_ID → INVESTOR_ID, SECURITY_ID, ORDER_ID

You can get these three attributes by following the chain: SELL_TRADE_ID → TRADE → ORDER_RECORD → {INVESTOR_ID, SECURITY_ID, ORDER_ID}. A single sell trade can be matched against multiple buy lots (so there can be multiple CG_ID rows with the same SELL_TRADE_ID), meaning it's not a superkey. Storing them here is just redundancy.

Violation 2 – HOLDING_DAYS → GAIN_TYPE

Under Indian capital gains tax rules, GAIN_TYPE is 100% determined by HOLDING_DAYS: under 365 days = SHORT_TERM, 365 or more = LONG_TERM. HOLDING_DAYS is not a superkey (many records can have the same holding period). If HOLDING_DAYS is corrected and GAIN_TYPE isn't updated, you get inconsistency.

Violation 3 – {BUY_PRICE, SELL_PRICE, QUANTITY} → GAIN_LOSS

GAIN_LOSS = (SELL_PRICE - BUY_PRICE) * QUANTITY. It's derived, so same story as TOTAL_CHARGES and UNREALISED_PNL.

Fix – Lossless Decomposition

Remove the redundant columns: INVESTOR_ID, SECURITY_ID, ORDER_ID (derive via SELL_TRADE_ID chain), GAIN_TYPE (compute with a CASE statement on HOLDING_DAYS), and GAIN_LOSS (compute from prices and quantity). All removals are lossless.

Corrected DDL:

```
CREATE TABLE CAPITAL_GAINS_RECORD (
  CG_ID            SERIAL PRIMARY KEY,
  BUY_TRADE_ID     INT REFERENCES TRADE(TRADE_ID) ON DELETE SET NULL,
  SELL_TRADE_ID    INT REFERENCES TRADE(TRADE_ID) ON DELETE SET NULL,
  BUY_PRICE        NUMERIC(15, 4),
  SELL_PRICE       NUMERIC(15, 4),
  QUANTITY         NUMERIC(15, 4),
  TAX_AMOUNT       NUMERIC(15, 2),
  HOLDING_DAYS     INT,
  SETTLEMENT_DATE  DATE,
  FINANCIAL_YEAR   VARCHAR(10)
  -- GAIN_TYPE = CASE WHEN HOLDING_DAYS < 365 THEN 'SHORT_TERM' ELSE 'LONG_TERM'
END
  -- GAIN_LOSS = (SELL_PRICE - BUY_PRICE) * QUANTITY
  -- INVESTOR_ID via: SELL_TRADE_ID -> ORDER_RECORD -> INVESTOR_ID
);
```

Table 22 – CLIENT_PLAN

Attributes: CLIENT_PLAN_ID (PK), INVESTOR_ID, PLAN_ID, START_DATE, END_DATE, IS_ACTIVE

Candidate Keys: {CLIENT_PLAN_ID}

FD	Dependency	Status
FD1	CLIENT_PLAN_ID → INVESTOR_ID, PLAN_ID, START_DATE, END_DATE, IS_ACTIVE	✓ (CK1)

✓ In BCNF – single FD, LHS is the primary key. Note: (INVESTOR_ID, PLAN_ID) could be a composite CK in some business models, but since the schema doesn't enforce a UNIQUE constraint on this pair, it's not formally a CK here.

Summary – BCNF Analysis of All 22 Relations

#	Table Name	Candidate Keys	BCNF?	Violation FD(s)
1	USER	{USER_ID}, {EMAIL}	✓	—
2	COMPLIANCE_OFFICER	{OFFICER_ID}, {EMPLOYEE_ID}	✓	—
3	INVESTOR	{INVESTOR_ID}, {PAN_NO}, {AADHAR_NO}	✗	DOB → AGE
4	KYC_DOCUMENT	{DOC_ID}, {DOC_NO}	✓	—
5	RETAIL_INVESTOR	{INVESTOR_ID}	✓	—

#	Table Name	Candidate Keys	BCNF?	Violation FD(s)
6	INSTITUTIONAL_INVESTOR	{INVESTOR_ID}, {CIN_NO}	✓	—
7	DEMAT_ACC	{DEMAT_ID}	✓	—
8	BANK_ACC	{BANK_ACC_ID}, {ACC_NO}	✗	IFSC_CODE → BANK_NAME, BRANCH_NAME
9	FUND_TRANSACTION	{TXN_ID}, {REFERENCE_NUMBER}	✓	—
10	BROKER	{BROKER_ID}, {USER_ID}, {SEBI_LICENSE_NO}	✓	—
11	BROKERAGE_PLAN	{PLAN_ID}	✓	—
12	BROKER_CLIENT	{BC_ID}, {INVESTOR_ID, BROKER_ID}	✓	—
13	PORTFOLIO	{PORTFOLIO_ID}	✓	—
14	SECURITY	{SECURITY_ID}, {TICKER}, {ISIN}	✓	—
15	EQUITY	{SECURITY_ID}	✓	—
16	MUTUAL_FUND	{SECURITY_ID}	✓	—
17	HOLDING	{HOLDING_ID}, {PORTFOLIO_ID, SECURITY_ID}	✗	{QTY, AVG_COST, CURR_VAL} → UNREALISED_PNL
18	TRADING_ACC	{TD_ID}	✓	—
19	ORDER_RECORD	{ORDER_ID}	✓	—
20	TRADE	{TRADE_ID}, {TRADE_REF}	✗	ORDER_ID → INVESTOR_ID, SECURITY_ID | Charges → TOTAL_CHARGES
21	CAPITAL_GAINS_RECORD	{CG_ID}	✗	SELL_TRADE_ID → ... | HOLDING_DAYS → GAIN_TYPE | Prices → GAIN_LOSS
22	CLIENT_PLAN	{CLIENT_PLAN_ID}	✓	—

Closing Remarks

What kind of violations were these?

All 5 violations fall into 2 common design anti-patterns:

- Derived / Computed Attributes stored with their source data – AGE (from DOB), UNREALISED_PNL (from qty and prices), TOTAL_CHARGES (sum of charge columns), GAIN_LOSS (from prices and qty), GAIN_TYPE (from HOLDING_DAYS). These can always be computed, so you should never store them in a normalized relation.

- Transitive redundancy via foreign-key chains – Storing INVESTOR_ID and SECURITY_ID in TRADE even though they're reachable via ORDER_RECORD. Since ORDER_ID is not a superkey in TRADE (one order can have many partial fill rows), this is a real BCNF violation.

Why are all decompositions lossless?

Each fix either (a) removes a derived attribute – lossless by definition since you can always recompute it – or (b) splits a relation on the PK of one of the resulting relations (like the BANK_ACC / BANK_BRANCH split on IFSC_CODE which becomes the PK of BANK_BRANCH). Both cases satisfy the lossless-join condition.

Practical note

Storing computed columns like UNREALISED_PNL or TOTAL_CHARGES is actually common in production systems for performance – usually maintained by DB triggers. But strictly from a normalization standpoint, they are BCNF violations. The correct academic answer is: store only irreducible facts, compute everything else.